

- Instruction name
- Instruction format (e.g. opcode | register | register. Something comparable to the R-type, I-type, J-type instruction formats given in MIPS)
- RTL description of instruction (e.g. $PC \leftarrow PC + 1$, $M(X) \leftarrow R(Y)$, etc.)
- Description of instruction in words
- For each instruction that is not obvious, include a note for when it is useful (e.g. ADD does not need an additional note. ShiftLeft might warrant a note. FP-Arith would definitely warrant a note.) Use your best judgement on which instructions need a note. (Tip: If any of the members of the group are unsure, it should have a note)

32 registers - 8 non-addressable

6 bit opcode

26 address bits

Word addressable

32 bit word

64 Instructions

ALU: Piper

MEM: Zach

Control:Brad

FP: Eldon Whiteley

Other: Harrison

000xxx - Other - 8 instructions

001xxx - Memory - 8 instructions

01xxxx - ALU - 16 Instructions

10xxxx - FP - 16 Instructions

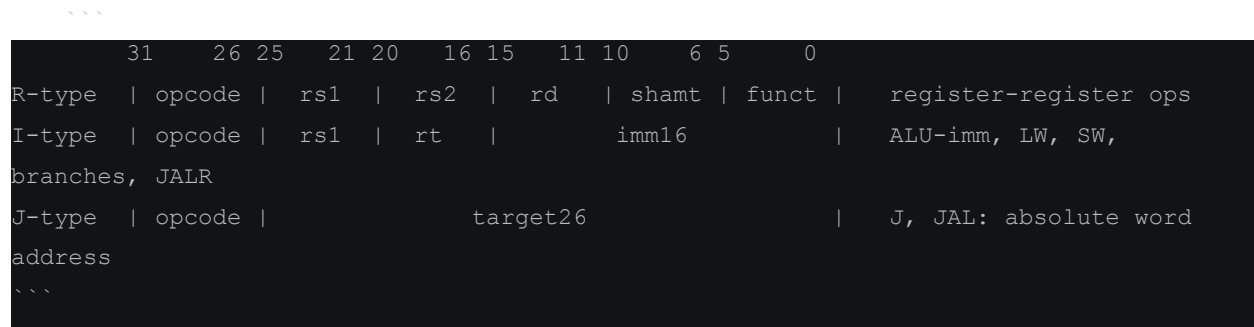
11xxxx - Control - 15 Instructions

111111 - Halt

Special Registers

- R0 - non-addressable, holds zero
- R1 - non-addressable, holds 1
- Flag Register (fr) - Negative, Zero, Less than, Equal
[0b0000 0000 0000 0000 0000 0000 0000 (n)(z)(l)(e)]

Instruction Formats



Other:

Name	Format	RTL Description	Description	Note
CLZ 000-000	Opcode [31-26] rs1[25-21] dc[20-16] rd[15-11] dc[11-0]	R[rd] <- clz(R[rs1]); PC++	Count the leading zeroes in an integer.	
CNTT0 000-001	Opcode [31-26] rs1[25-21] dc[20-16] rd[15-11] dc[11-0]	R[rd] <- ctz(R[rs1]); PC++	Count the trailing zeros in an integer.	
POP 000-010	Opcode [31-26] rs1[25-21] dc[20-16] rd[15-11] dc[11-0]	R[rd] <- pop(R[rs1]); PC++	Count the number of active bits in an integer.	
BYTR 000-011	Opcode [31-26] rs1[25-21] dc[20-16] rd[15-11] dc[11-0]	R[rd] <- R[rs1][7:0] R[rs1][15:8] R[rs1][23:16] R[rs1][31:24]; PC++	Reverse the byte order in an integer.	
SEXTD 000-100	Opcode [31-26] rs1[25-21] dc[20-16] rd[15-11] dc[11-0]	R[rd] <- SEXTD(R[rs1][15:0]); PC++	Sign extend an integer.	
000-101				

000-110				
NOOP 000-111	Opcode [31-26], dc[26-0]	PC++	No op-do nothing.	

Memory:

Code	Name	Format	RTL Description	Description	Note
001-000	Load (LD)	I-type	$R[rt] = M[rs + \text{Immediate}]$ (31 - 0)	Loads a word from mem to a register.	
001-001					
001-010	Store (ST)	I-type	$M[rs + \text{Immediate}] = R[rt]$ (31 - 0)	Stores a word from a target register to mem	
001-011					
001-100	Move (MOV)	I-type	$R[rt] = R[rs]$	Loads the register source value into the register targe	
001-101	Load Register (LDR)	I-type	$R[rt] = \text{Immediate}$	Loads an immediate value into a register	
001-110					
001-111					

ALU:

Name	Format	RTL Description	Description	Note
010000 - ADD	R-Type	$R[rt] \leftarrow R[rs] + R[rt]$, PC++	Adds R1 to R2	
010001 - SUB	R-Type	$R[rt] \leftarrow R[rs] - R[rt]$, PC++	Subtracts R2 from R1	
010010 - RSB	R-Type	$R[rt] \leftarrow R[rt] - R[rs]$, PC++	Subtracts R1 from R2	
010011 - INC	R-Type	$R[rt] \leftarrow R[rs] + 1$, PC++	Increments R1 by 1	
010100 - DEC	R-Type	$R[rt] \leftarrow R[rs] - 1$, PC++	Decrements R1 by 1	
010101 - RS	R-Type	$R[rt] \leftarrow R[rs] \gg 1$, PC++	Shifts the bits of R1 to the right 1	
010110 - LS	R-Type	$R[rt] \leftarrow R[rs] \ll 1$, PC++	Shifts the bits of R1 to the left 1	
010111 - AND	R-Type	$R[rt] \leftarrow R[rs] \text{ AND } R[rt]$, PC++	ANDs R1 and R2	
011000 - OR	R-Type	$R[rt] \leftarrow R[rs] \text{ OR } R[rt]$, PC++	ORs R1 and R2	
011001 - NAND	R-Type	$R[rt] \leftarrow R[rs] \text{ NAND } R[rt]$, PC++	NANDs R1 and R2	
011010 - NOR	R-Type	$R[rt] \leftarrow R[rs] \text{ NOR } R[rt]$, PC++	NORs R1 and R2	
011011 - XOR	R-Type	$R[rt] \leftarrow R[rs] \text{ XOR } R[rt]$, PC++	XORs R1 and R2	
011100 - NOT	R-Type	$R[rt] \leftarrow \text{not } R[rs]$, PC++	Inverts all bits of R1	
011101 - ADDI	I-Type	$R[rt] \leftarrow R[rs] + \text{Immediate}$, PC++	Add an immediate to a register value, store in destination	

011110 - SUBI	I-Type	$R[rt] \leftarrow R[rs] - \text{Immediate}, PC++$	Subtract an immediate from a register value, store in destination	
011111 - SFLIP	R-Type	$R[rt] \leftarrow \neg(R[rs]), PC++$	Flip a value from positive to negative.	

Floating Point:

Code	Name	Format	Bit Layout	RTL Description	Description	Note
10 - 0000	FLW	I-Type	opcode rs1 rt imm16	$R[rt] = M[R[rs1] + \text{SignExt}(\text{imm16})]$	Load a floating point word from memory	
10 - 0001	FSW	I-Type	opcode rs1 rt imm16	$M[R[rs1] + \text{SignExt}(\text{imm16})] = R[rt]$	Store a floating point word in mem	
10 - 0010	FADD	R-Type	opcode rs1 rs2 rd 00000 000000	$R[rd] = R[rs1] + R[rs2]$	Add two floating point values	
10 - 0011	FSUB	R-Type	opcode rs1 rs2 rd 00000 000000	$R[rd] = R[rs1] - R[rs2]$	Subtract two floating point values	
10 - 0100	FMULT	R-Type	opcode rs1 rs2 rd 00000 000000	$R[rd] = R[rs1] * R[rs2]$	Multiply two floating point values	
10 - 0101	FDIV	R-Type	opcode rs1 rs2 rd 00000 000000	$R[rd] = R[rs1] / R[rs2]$	Divide two floating point values	
10 - 0110	FFMA	R-Type	opcode rs1 rs2 rd rs3 000000	$R[rd] = (R[rs1] * R[rs2]) + R[rs3]$	Fused multiply add	Truncate
10 - 0111	FSQRT	R-Type	opcode rs1 00000 rd 00000 000000	$R[rd] = \text{sqrt}(R[rs1])$	Find the square root of an FP value	
10 - 1000	FNEG	R-Type	opcode rs1 00000 rd 00000 000000	$R[rd] = R[rs1] \wedge 0x80000000$	Sets the sign bit high	

10 - 1001	FABS	R-Type	opcode rs1 00000 rd 00000 000000	$R[rd] = R[rs1] \& 0x7FFFFFFF$	Clears the sign bit	
10 - 1010	FLT	R-Type	opcode rs1 rs2 00000 rs3 000000	$FR[1] = (R[rs1] < R[rs2]) ? 1 : 0$	Sets a bit if rs1 is less than rs2	
10 - 1011	FEQ	R-Type	opcode rs1 rs2 00000 00000 000000	$FR[0] = (R[rs1] == R[rs2]) ? 1 : 0$	Sets a bit if two FP vals are equal	
10 - 1100	FMAX	R-Type	opcode rs1 rs2 rd 00000 000000	$R[rd] = (R[rs1] > R[rs2]) ? R[rs1] : R[rs2]$	Store the max of two FP values	
10 - 1101	FTOIW	R-Type	opcode rs1 00000 rd 00000 000000	$R[rd] = (int)R[rs1]$	Convert a floating point value to int word	Truncate
10 - 1110	IWTOF	R-Type	opcode rs1 00000 rd 00000 000000	$R[rd] = (float)R[rs1]$	Convert a int word to a floating point value	
10 - 1111	FMOV	R-Type	opcode rs1 00000 rd 00000 000000	$R[rd] = R[rs1]$	Move a floating point value to another register	

Control:

Name	Format	RTL Description	Description	Note
11 - 0000 BEQ	R	$r1 == r2 ? \text{dest} : \text{PC}+$	Branch to destination if $op1 == op2$, otherwise continue	
11 - 0001 BLT	R	$r1 < r2 ? \text{dest} : \text{PC}+$	Branch to destination if $op1 < op2$, otherwise continue	
11 - 0010 BLE	R	$r1 \leq r2 ? \text{dest} : \text{PC}+$	Branch to destination if $op1 \leq op2$, otherwise continue	
11 - 0011 BGT	R	$r1 > r2 ? \text{dest} : \text{PC}+$	Branch to destination if $op1 > op2$, otherwise continue	
11 - 0100 BGE	R	$r1 \geq r2 ? \text{dest} : \text{PC}+$	Branch to destination if $op1 \geq op2$, otherwise continue	
11 - 0101 BNE	R	$r1 \neq r2 ? \text{dest} : \text{PC}+$	Branch to destination if $op1 \neq op2$, otherwise continue	
11 - 0110 JUMP	J	$\text{PC} = \text{dest}$	Jump to destination	
11 - 0111 JUMPR	J	$\text{PC} = r1$	Jump to destination in register $op1$	
11 - 1000 JLINK	J	$\text{PC} = r1, r2 = \text{PC}+$	jump to destination in op	

			1, save next sequential instruction in op2	
11 - 1001 BEXT (TENTATIVE)	J		Like jump, only executed if coming from positive branch test. Used for extending reach	
11 - 1010 IN (TENTATIVE)	IN (TENT.)	$r1 = IN$	Read word input to op1	
11 - 1011 OUT (TENTATIVE)	OUT (TENT.)	$OUT = r1$	output from op1	
11 - 1100 RET (TENTATIVE)	RET (TENT.)	$PC = r2$	Quit subroutine and return to previous	
11 - 1101 FILL (TENTATIVE)	FILL (TENT.)	$r1 = r1 \parallel 2^{32}$	fill all bits with 1 in op 1	
11 - 1110 CLR (TENTATIVE)	CLR (TENT.)	$r1 = r1 \&\& 0$	clear all bits to 0 in op 1	
11 - 1111 HALT		HALT	end execution	